



Мельников Денис

☎ +7 931 511 58 76
✉ melnikov.denis.aleksandrovich@gmail.com
🐙 [@daikofnetu](#)
🌐 [Dinichthys](#)
📍 Москва, Россия

Образование

Бакалавриат Второй курс, Средний балл: 8.65	Сен. 2025 – Настоящее время
Физтех-школа радиотехники и компьютерных технологий	
Московский физико-технический институт	Долгопрудный, Россия

Дополнительные курсы

Курс Ильи Рудольфовича Дединского по языку C и языкам ассемблера	Сен. 2024 – Май 2025
Курс Ильи Рудольфовича Дединского по языку C++	Сен. 2025 – Май 2026
Введение в LLVM IR от Сергея Лисицына (МФТИ)	Фев. 2026 – Март 2026
Введение в архитектуру вычислительных систем от организации СБЕР	Фев. 2026 – Май 2026
Введение в тензорные компиляторы от организации СБЕР	Фев. 2026 – Май 2026

Опыт работы

Опыт работы в команде разработчиков компилятора Go	Июль - Сентябрь 2025
Стажёр (ассистент-инженер) в компании Huawei на полной ставке. Я дорабатывал определённый оптимизационный пасс в компиляторе, чтобы он рассматривал больше случаев. По итогам моей стажировки мой коммит в официальный репозиторий компилятора был принят и изменения внесены.	

Опыт преподавания

Опыт менторства на курсе Ильи Рудольфовича Дединского	Сен. 2025 - Май 2026
Я работал ментором на курсе Ильи Рудольфовича Дединского по языку C и языкам ассемблера для первокурсников на протяжении полного года обучения.	

Проекты

Compiler C, Cmake, NASM	Compiler on GitHub
Цель проекта: Реализовать компилятор для собственного языка программирования под собственную архитектуру, а также NASM. Front-end компилятора разбивал текст программы на лексемы, по которым строилось абстрактное синтаксическое дерево (AST) с помощью рекурсивного спуска. Затем оно переводилось в промежуточное представление. Далее оно оптимизировалось Middle-end'ом, в котором производились следующие оптимизации: • Dead code elimination: убирал код, который не был достижим. • Common subexpression: результат одинаковых подвыражений вычислялся единожды. • Unused variables: неиспользуемые переменные удалялись из программы. Затем Back-end транслировал IR в ассемблерные инструкции для функционального симулятора, который впоследствии преобразовывал их в машинный код и выполнял.	
LLVMPass C++, Cmake	LLVMPass on GitHub
Цель проекта: Реализовать пасс, который бы строил CFG и DFG, а также инструментировал LLVMIR логирующими инструкциями, а затем собирал полученный профиль и по нему на графе отображал цветами и количеством вызовов горячие и холодные функции и базовые блоки в них.	

Цель проекта: Создать физический конструктор разных объектов с возможностью отражения и преломления в них света, а также возможность добавления плагинов. Таким образом, в конечном варианте было 2 плагина - плагин для графической библиотеки (путём его замены можно было перейти с SFML на SDL, например, не переписывая ничего), а также плагин для дорисовки (функционал для рисования различных фигур поверх экрана).

Реализации моих плагинов можно найти в репозиториях - DR4Backend (плагин графической библиотеки) и GeomPrimitiveBackend (плагин дорисовки).

Цель проекта: Реализовать функциональный симулятор процессора с тремя компонентами:

- Ассемблер: Транслирует ассемблерные инструкции в бинарное представление.
- Дизассемблер: Восстанавливает ассемблерные инструкции из бинарного представления.
- Процессор: Исполняет инструкции, описанные с помощью бинарного представления.

Цель проекта: Реализовать упрощённую версию функции языка C "printf" на языке ассемблера с поддержкой следующих спецификаторов форматной строки: %c, %s, %d, %b, %o, %x, и %f. Определение спецификатора происходило с помощью JUMP-таблицы.

Дополнительные реализованные возможности:

- Вывод в файл: Поддерживается вывод в файл.
- Цветной вывод:
 - * При записи в стандартный поток вывода для смены цвета использовались escape-последовательности.
 - * При записи в файл для задания цвета текста были использованы теги HTML (например,)
- Числа с плавающей точкой: Использовался спецификатор %f для записи чисел с плавающей точкой.

Цель проекта: Оптимизировать программу отрисовки множества Мандельброта разными способами.

Было использовано три метода оптимизации:

- Массивы: Использование массивов особым образом позволило компилятору заменить операции над ними на SIMD-инструкции.
- SIMD Инструкции: Использование интринсиков позволило оптимизировать программу, потому что теперь стали исполняться инструкции сразу над набором данных.
- Раскрутка цикла: Раскрутка цикла помогла избежать зависимости по данным между инструкциями, из-за чего конвейер всегда был загружен.

Интересы

Компиляторы
Оптимизации
Нейронные сети
Операционные системы

Hard Skills

Языки программирования: C, C++, Assembly (NASM, TASM, x86-64), Python, GoLang
Инструменты: CMake, Make, IDA, Ghidra, Radare2, edb, gdb

Языки

Русский: Родной
Английский: B1

Soft Skills

Коммуникабельный
Уверенный в себе
Ответственный